

members. Similarly as classes, mixins use a closed form of the mixin declaration where the entire declaration is provided within the `define(name, config)` method. In the method, `name` is a fully-qualified mixin name, and `config` is a configuration map with various sections defined by keys:

```
• Osp.Mixin.define("name",
• {
•   include: [SuperMixins],
•   members:
•   {
•     prop1: "foo",
•     meth1: function() { },
•     meth2: function() { }
•   }
• });
```

bada user interface

The normal setup for a simple bada web application UI is a full-screen `Osp.Ui.Controls.Form` container within an `Osp.Ui.Controls.Frame` container that creates the top-level window. The form in turn contains the indicator bar, header, footer, and UI controls.

There can only be one frame per application. However, there can be multiple forms, though only one form can be active at a time.

The `Ui` namespace provides a meta-model for UX development. You must adhere to the model in order to use the functionalities of the `Ui` namespace.

UI controls

UI controls are user interface elements, such as buttons, lists, and text fields, that are commonly used for organizing content on the device screen, providing means for user navigation or notifying about application status changes.

The `Osp.Ui.Control` class is the abstract base class of all user interface elements. It encapsulates the native display functions for the underlying windowing system, and provides the infrastructure necessary for the elements to respond to user inputs. It also determines how the key events are dispatched and processed.

Additionally, the `Control` class defines the following properties that are shared by all controls and allow for a higher degree of control handling:

- ▶ Position and size
- ▶ Drawing logic